Software Engineering Methodologies & Scrum

Introduction

In this lecture, we examine why traditional plan-driven (Waterfall) development fails in dynamic environments and introduce the empirical Agile paradigm.

You will learn the core mechanics of the Scrum framework—including its 3 pillars, team accountabilities, ceremonies (events), and artifacts—alongside practical estimation and forecasting tools like story points, Planning Poker, and velocity tracking.

1. Methodological Foundations

Understanding the Philosophy of Software Development & Process Control Models

Software vs Engineering

🏛 Civil & Mechanical Engineering

Physical & Deterministic: Governed by established physical laws (statics, dynamics, material science).

Strict Replication: Once validated through structural mechanics, execution centers on precise physical construction.

High Alteration Cost: Physical modifications post-construction incur exponential cost penalties.

</> Software Development

Intellectual & Invisible: Systems are highly malleable, non-linear, and abstract by nature.

Low Modification Cost: Modifying digital assets appears cheap, prompting ongoing stakeholder revisions.

Continuous Mutation: Software environments and market expectations evolve rapidly during production.

Defined vs Empirical Process Control

Defined Process Control

Core Premise: Production process is fully understood, predictable, and strictly repeatable.

Input Control: Tight control over input parameters guarantees consistent output standards.

Domain Fit: Manufacturing assembly lines, low-complexity or safety-critical hardware systems with frozen specs.

Empirical Process Control

Core Premise: Process is complex, partially unmapped, and subject to rapid unexpected change.

Three Pillars: Relies on continuous observation, inspection, and ongoing adaptation.

Domain Fit: Modern software development where user requirements and technologies shift dynamically.

2. Failure of Plan-Driven Models

Analyzing the architectural vulnerabilities of traditional Waterfall SDLC in volatile environments

The Requirement Volatility Problem

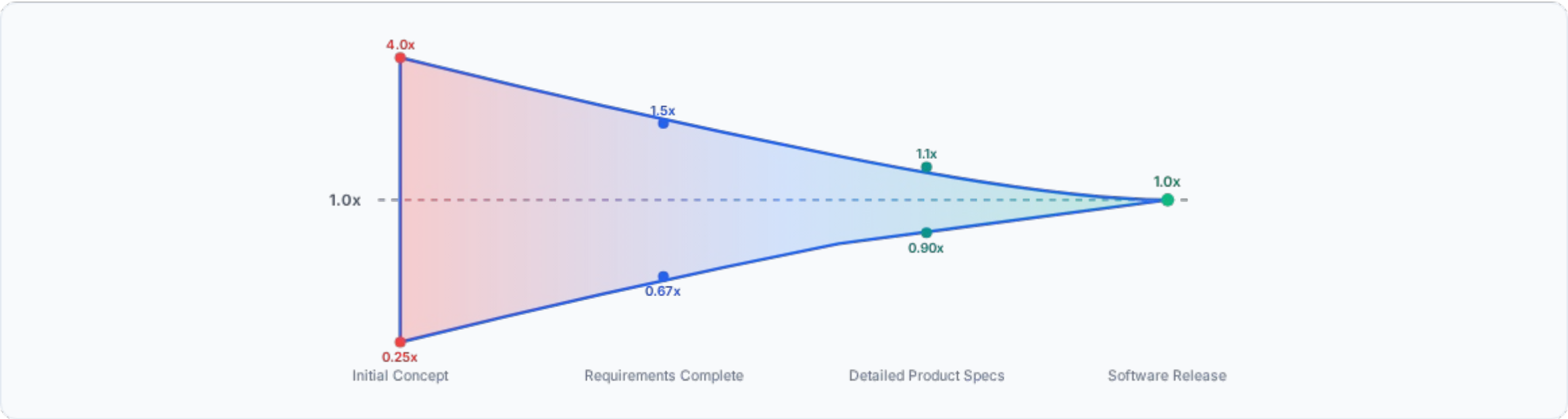
The Fallacy of Frozen Specs

Plan-driven frameworks assume software requirements can be completely discovered and locked prior to implementation.

In practice, requirements continuously evolve because: -

- **Stakeholder Blind Spots:** End-users rarely understand their actual operational needs until interacting with a working application interface.
- **Dynamic Market Drivers:** External regulatory shifts, competitor moves, and market pressures continuously alter business priorities mid-project.
- **Technical Discoveries:** Architectural bottlenecks and unpredicted platform constraints naturally surface during implementation.
- **The Frozen Spec Fallacy:** Attempting to lock requirements upfront creates a widening gap between delivered software and true business value.

Boehm's Cone of Uncertainty



Plan-driven frameworks commit to fixed budgets and delivery dates at project kickoff – the exact point when estimation variance (0.25x to 4.0x) and uncertainty are at their absolute peak.

The Late Risk Accumulation Profile

Deferred Risk Discovery

In traditional sequential models, integration and system testing are performed near the tail end of the project life cycle.

Consequently, operational risk accumulates silently throughout development and peaks during final verification.

Exponential Rework Expenses

If core architectural defects or security flaws are identified during late testing, resolving them requires cascading changes.

Teams must refactor design, rewrite code, and update documentation across all layers, leading to major budget overruns.

3. The Agile Paradigm

Replacing static administrative controls with empirical feedback loops and continuous delivery

The 4 Agile Manifesto Values



Individuals & Interactions

Over processes and heavy tooling to foster direct communication and rapid alignment.



Working Software

Over comprehensive documentation as the primary measure of real project progress.



Customer Collaboration

Over rigid contract negotiation to co-create solutions alongside stakeholders.



Responding to Change

Over following a fixed plan to capitalize on emerging operational insights.

12 Principles of Agile

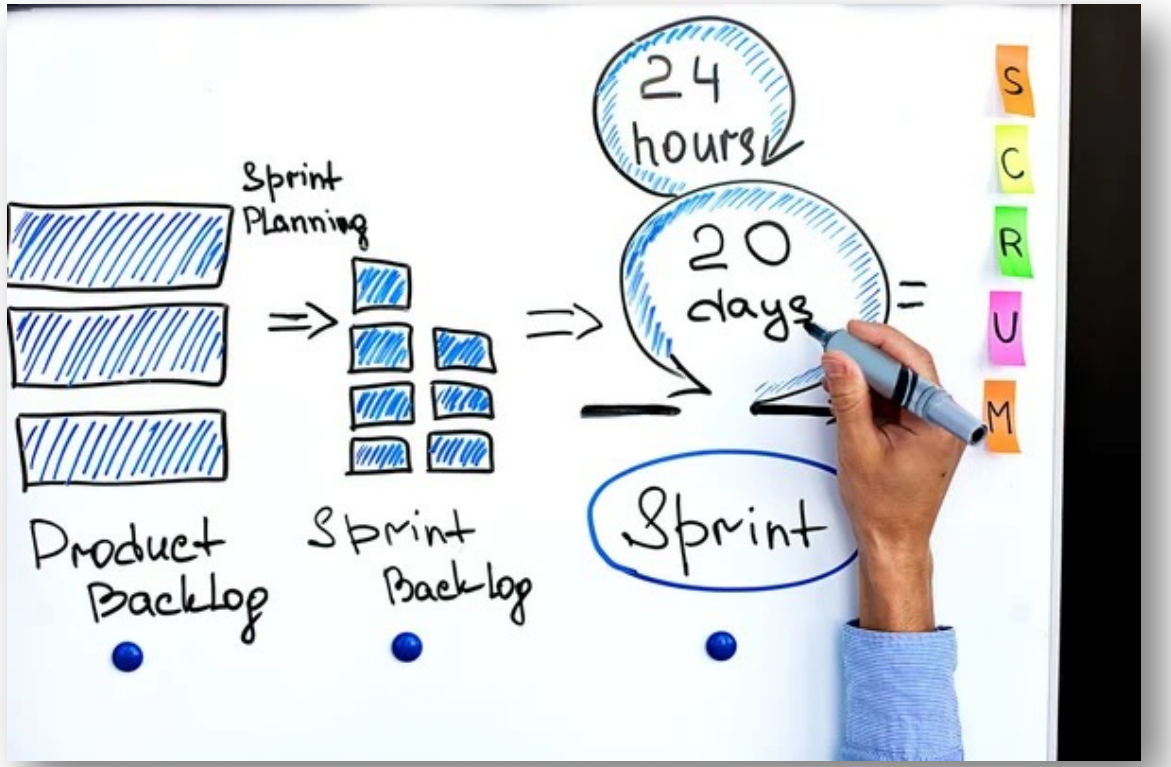
- **Customer Satisfaction:** Highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- **Welcome Changing Requirements:** Harness change for the customer's competitive advantage, even late in development.
- **Frequent Delivery:** Deliver working software frequently, from a couple of weeks to a couple of months, preferring shorter timescales.
- **Cross-Functional Collaboration:** Business people and developers must work together daily throughout the project.
- **Motivated Individuals:** Build projects around motivated individuals; give them the environment and support they need, and trust them to get the job done.



Source: leadingagileteams.com

12 Principles of Agile

- **Face-to-Face Communication:** The most efficient and effective method of conveying information is face-to-face conversation.
- **Working Software as Progress:** Working software is the primary measure of progress.
- **Sustainable Development:** Promoters, developers, and users should be able to maintain a constant pace indefinitely.
- **Technical Excellence:** Continuous attention to technical excellence and good design enhances agility.
- **Simplicity:** The art of maximizing the amount of work not done is essential.



Source: www.linkedin.com

12 Principles of Agile

- **Self-Organizing Teams:** The best architectures, requirements, and designs emerge from self-organizing teams.
- **Regular Reflection:** At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.



Source: www.accelo.com

Plan-Driven vs. Agile Frameworks

Architectural Vector	Plan-Driven (Waterfall)	Agile Frameworks (Scrum/Kanban)
Core Assumption	Predictable environment & deterministic specs	Dynamic environment & evolving scope
Process Control Mode	Defined process control	Empirical process control (Inspect & Adapt)
Primary Metric	Milestone compliance & sign-offs	Working software & validated value
Cost of Change	Escalates exponentially over time	Kept low through iterative refinement
Stakeholder Role	Upfront requirements & end testing	Continuous active collaboration
Risk Profile	Late risk discovery during integration	Early and continuous risk mitigation

4. Empirical Process Control in Scrum

Replacing predictive forecasting with continuous observation, inspection, and rapid adaptation

The Three Pillars of Scrum

1. Transparency

Visibility of Work: Significant aspects of the process must be visible to those responsible for the outcome.

Requires a single, shared standard—such as a common Definition of Done (DoD)—so viewers share a unified understanding of status.

2. Inspection

Cadence of Review: Scrum users must frequently inspect artifacts and progress toward goals to detect undesirable variances.

Inspections are integrated into regular Scrum events without impeding day-to-day progress or creating administrative bloat.

3. Adaptation

Pivot & Adjustment: If inspection reveals process deviations outside acceptable limits, the process or artifacts must be adjusted immediately.

Adaptation must happen as quickly as possible to minimize further variance and maximize delivered value.

Empiricism vs. Predictive Control

Theoretical & Predictive Management

Linear Planning: Assumes software construction follows a deterministic path mapped out months in advance.

Status Reporting: Focuses heavily on vanity metrics, document sign-offs, and compliance with original projections.

Risk Profile: High vulnerability to market changes, late discovery of architectural defects, and misalignment with actual user needs.

Empirical Process Execution

Iterative Steering: Decisions are made based on hard data, observed reality, and verified working software increments.

Feedback Loops: Short execution cycles (Sprints) force early detection of risks, technical blockers, and scope shifts.

Value Maximization: Enables teams to pivot gracefully without discarding months of sunk labor or architecture.

5. The Three Scrum Team Accountabilities

Establishing clear separation of responsibilities: Optimizing Value, Flow, and Technical Quality

The Scrum Team

Product Owner (PO)

Focus: What to build & Why.

Accountable for maximizing product value and effective Product Backlog management. Single voice representing stakeholders and market requirements.

Developers

Focus: How to build & Deliver.

Accountable for creating a usable, testable increment every Sprint. Self-organizing around technical implementation, quality standards, and DoD.

Scrum Master (SM)

Focus: How to optimize flow & Process.

Accountable for establishing Scrum as defined in the Scrum Guide. Servant-leader who coaches the team, removes impediments, and fosters self-management.

The Product Owner (PO) Accountability

Core Responsibilities

Backlog Management: Developing and explicitly communicating the Product Goal and ordering Backlog items.

Value Optimization: Deciding release dates, evaluating return on investment (ROI), and defining acceptance criteria.

Decisive Autonomy: The entire organization must respect PO decisions reflected in the Product Backlog content and ordering.

Common PO Anti-Patterns

Pass-Through Proxy: Lacking authority to make real decisions, acting only as a message passer for committee demands.

Technical Dictator: Overriding developer estimates or dictating *how* technical implementation should be written.

Absentee Owner: Unavailable during Sprints, causing developer blockages during requirement clarification.

The Scrum Master (SM) Accountability

Core Responsibilities

Coaching the Organization: Helping stakeholders understand empirical software development and cross-team alignment.

Facilitating Flow: Ensuring all Scrum events take place, are productive, positive, and kept within timeboxes.

Removing Obstacles: Actively stripping away organizational and technical impediments beyond team control.

Common SM Anti-Patterns

Scribe & Administrator: Functioning strictly as a JIRA clerk or meeting scheduler rather than an agile coach.

Directive PM / Taskmaster: Assigning tasks to developers, demanding status reports, or managing individual output.

Process Dogmatist: Rigidly enforcing rules without understanding the underlying principles of empiricism.

The Developers: Team Attributes

- **Self-Organizing & Autonomous:** Developers decide *how* to turn Product Backlog items into increments of value. No external authority dictates work assignment.
- **Cross-Functional Capability:** The team collectively possesses all skills necessary—architecture, UI/UX, coding, testing, and DevOps—to create a usable increment.
- **No Titles or Sub-Teams:** Everyone is a "Developer" regardless of primary specialization. Specialized sub-teams (e.g., testing team, database team) within the Scrum Team are strictly prohibited.
- **Collective Ownership & Accountability:** Success or failure belongs to the whole team. Individual accountability is replaced by shared commitment to the Sprint Goal.

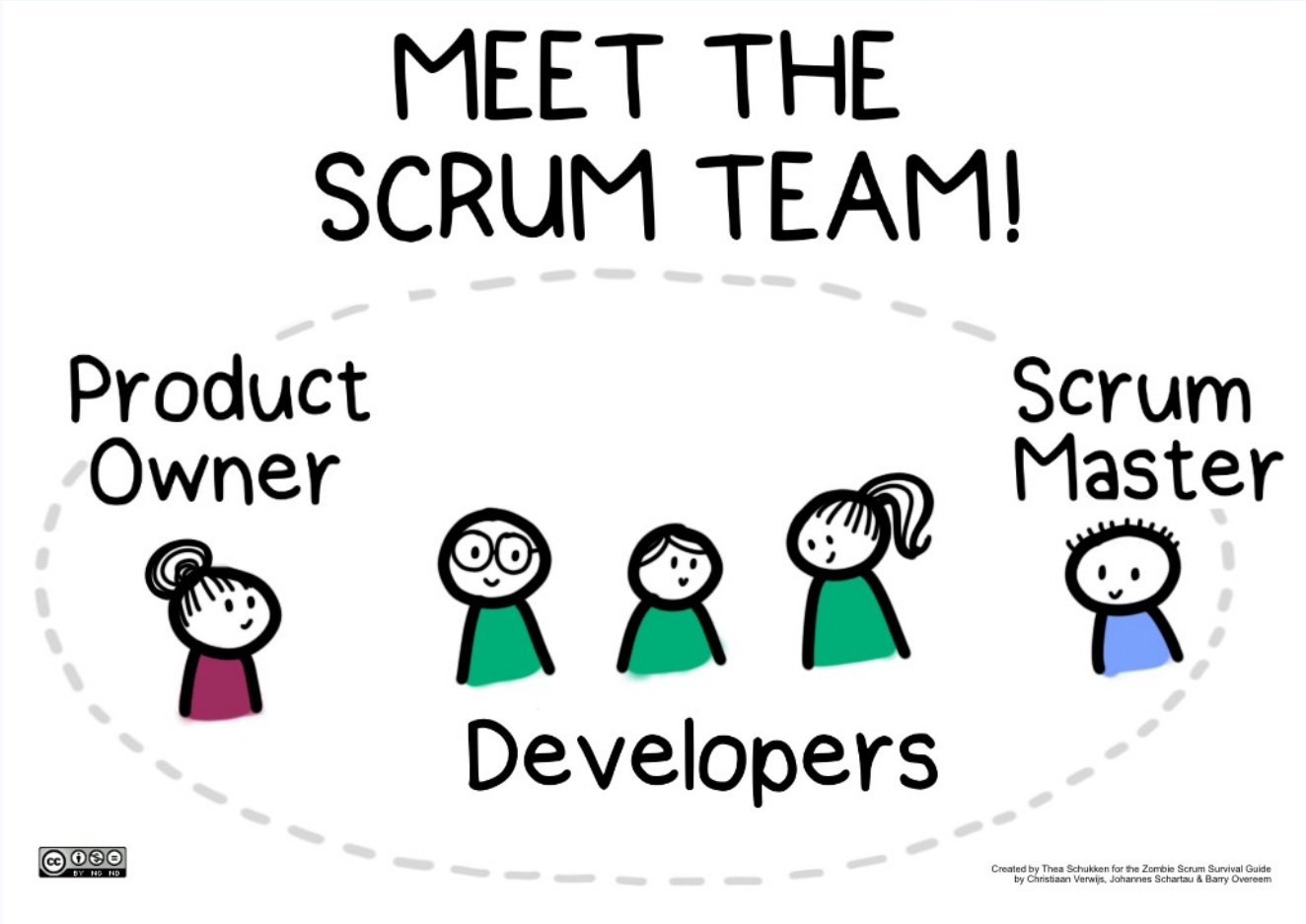
Scrum Team Collaboration

Synergy of the Team

The Scrum Team is intentionally structured to balance competing forces in software engineering:

The Product Owner pushes for maximum *Value* and market impact. The Developers safeguard *Quality* and architectural integrity. The Scrum Master optimizes *Flow* and team sustainability.

When all three accountabilities operate in harmony, software delivery becomes fast, predictable, and adaptive.



Source: medium.com

6. The Four Scrum Ceremonies

Timeboxed events with opportunities for inspection and adaptation while minimizing unnecessary meetings.

Sprint Planning

Core Concepts

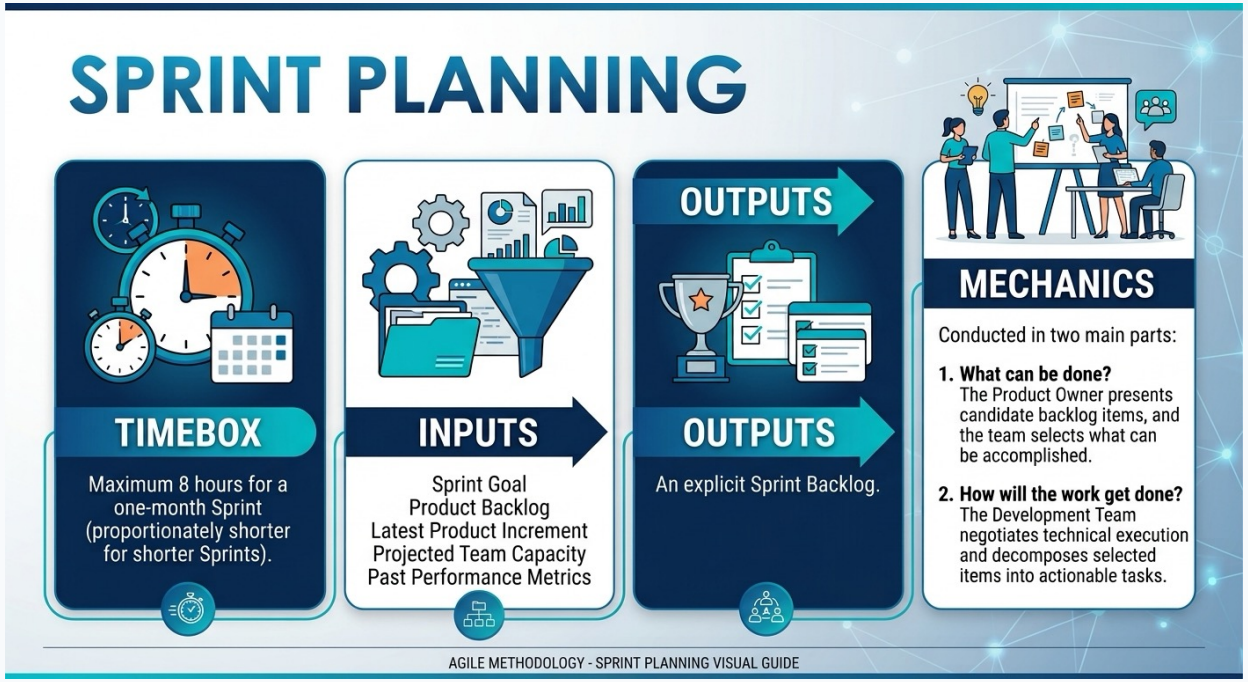
Timeboxed: Maximum 8 hours for a one-month Sprint

Input: Sprint Goal, product Backlog, latest product Increment, projected team capacity, and past performance metrics .

Output: Sprint Backlog.

The Process:

1. The Product Owner presents candidate backlog items, and the team selects what can be accomplished.
2. The Development Team negotiates technical execution and decomposes selected items into actionable tasks.



Source: Generated by Gemini

Scrum Execution & Daily Scrum

Core Concepts

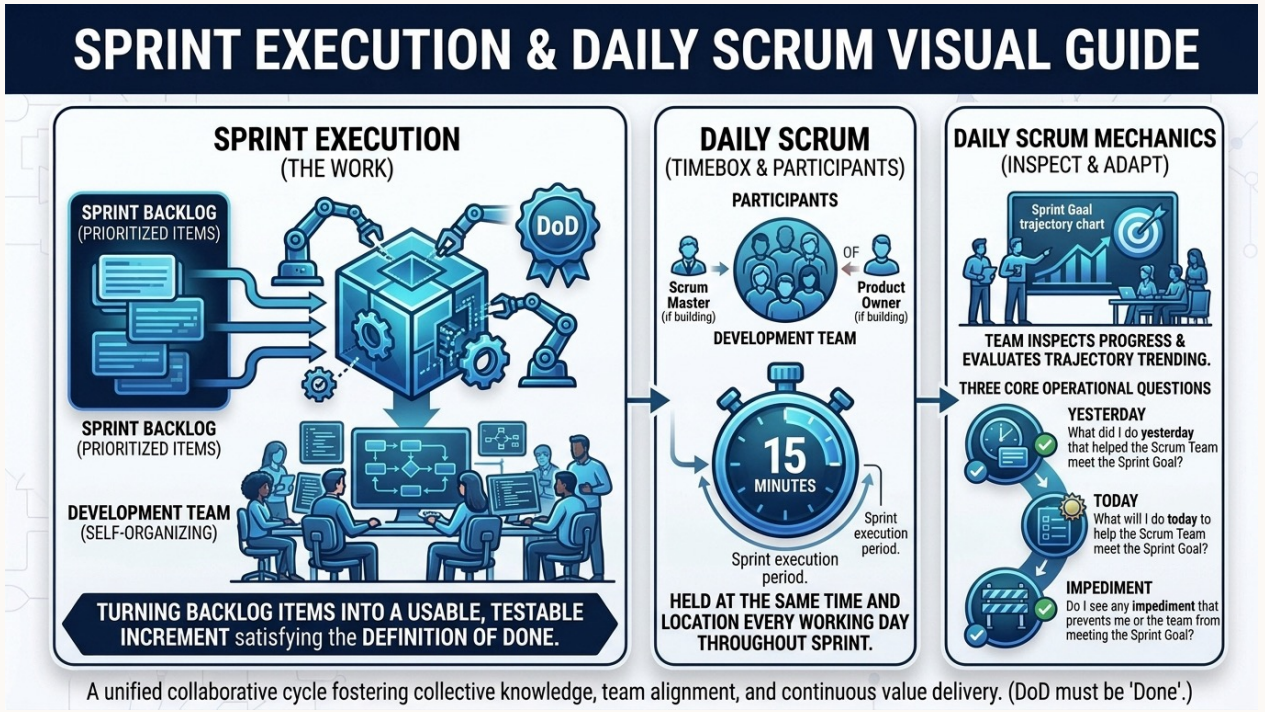
Sprint Execution: Implementation phase where the Development Team self-organizes to turn Sprint Backlog items into Increment satisfying the DoD

Daily Scrum Timeboxed: 15 minutes (at the same time and location everyday)

Participants: Development Team.

Daily Scrum Process: Inspects progress toward the Sprint Goal and evaluates how their trajectory is trending. Members answer three questions:

1. *What did I do yesterday?*
2. *What will I do today?*
3. *Do I see any impediment?*



Source: Generated by Gemini

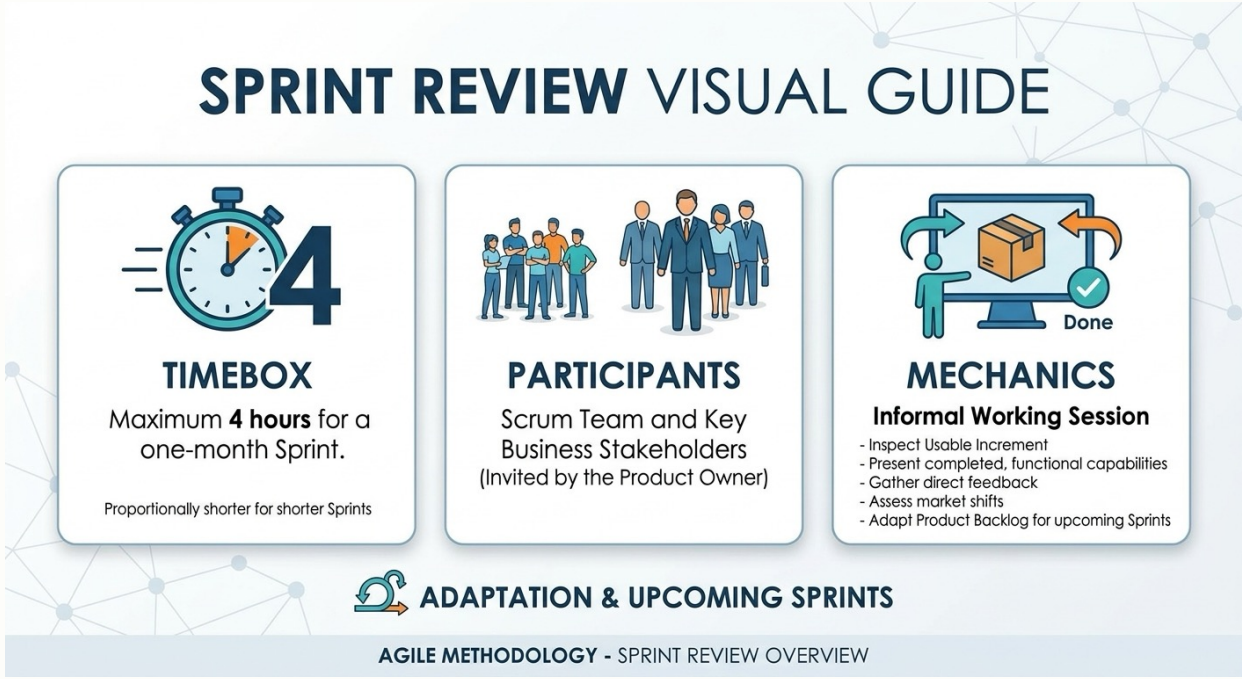
Sprint Review

Core Concepts

Timeboxed: Maximum 4 hours for a one-month Sprint

Participants: Scrum Team and key business stakeholders invited by the Product Owner .

The Process: To inspect the usable increment produced during the Sprint. The team presents completed, functional capabilities to gather direct feedback, assess market shifts, and adapt the Product Backlog for upcoming Sprints.



Source: Generated by Gemini

Sprint Retrospective

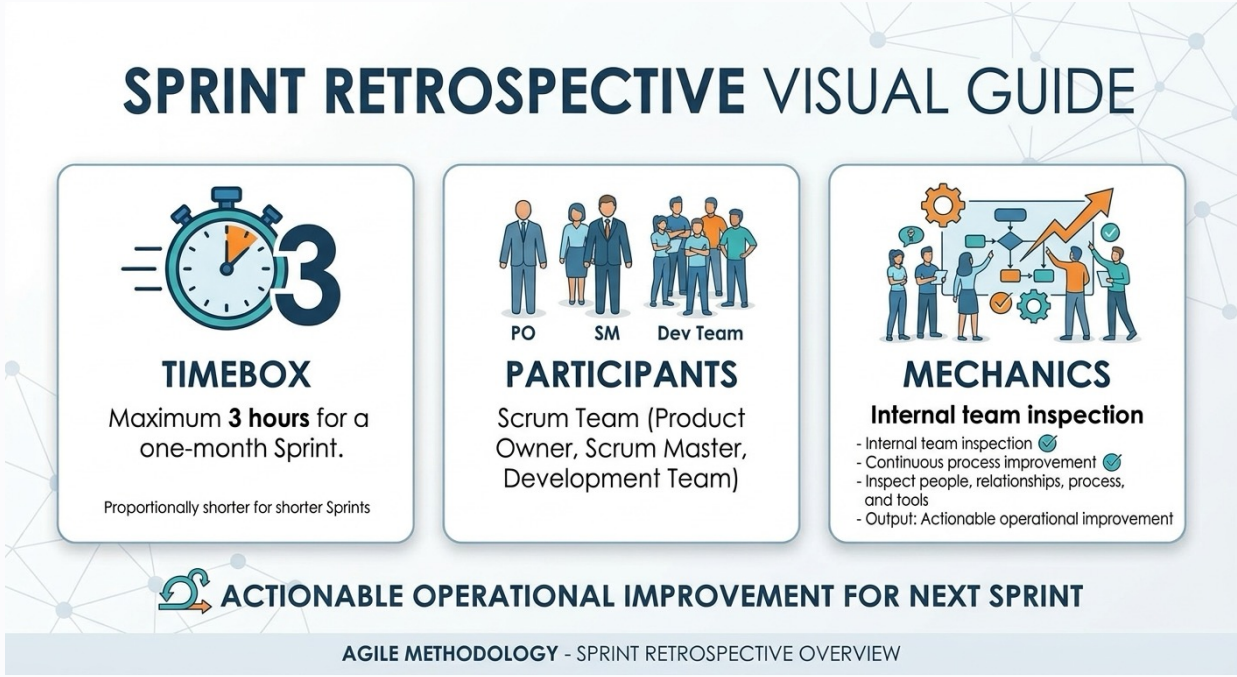
Core Concepts

Timeboxed: Maximum 3 hours for a one-month Sprint

Participants: Scrum Team

The Process: Inspection focused on continuous process improvement. Inspects how the last Sprint went regarding people, relationships, process, and tools.

Output: At least one actionable operational improvement to implement in the next Sprint



Source: Generated by Gemini

7. The Three Scrum Artifacts

Scrum artifacts represent work or value clearly, allowing everyone to inspect progress and adjust using the same facts

The Artifacts



Product Backlog

Definition: An ordered list of product requirements. It is the sole source of requirements.

Characteristics: It is dynamic and constantly evolving

Refinement: The ongoing process in which the Scrum team add detail, estimates, and order to items in the backlog



Sprint Backlog

Definition: Selected Product Backlog items for the Sprint, plus a plan for delivering the product Increment for the Sprint.

Characteristics: Development Team adds, updates, or removes technical tasks throughout the process.



Increment

Definition: The sum of all the Product Backlog items completed during this Sprint and the previous Sprints.

Characteristics: be in usable condition regardless of whether the Product Owner decides to release it to production immediately.

8. Definition Of Done

The DoD is a formal, shared understanding of what quality standards code must meet to considered a completed product Increment

Definition of Done (DoD)

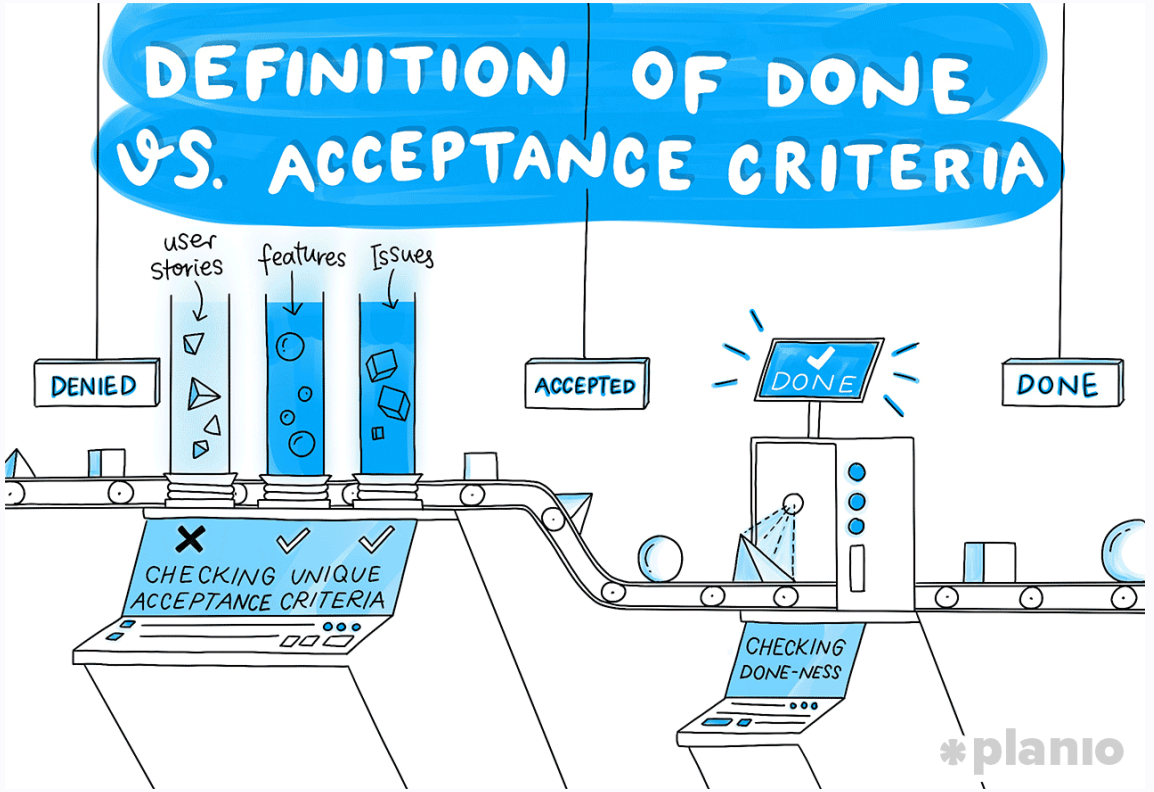
Core Concepts

Purpose: Ensures transparency so that when an item is declared "Done," everyone understands what was completed and what quality measures were validated.

DoD vs. Acceptance Criteria:

- **Acceptance Criteria:** Unique to each individual Product Backlog item
- **Definition of Done:** Applies uniformly to *all* backlog items

Incomplete DoD: If an item does not satisfy the DoD, it cannot be demonstrated at the Sprint Review or released; it must return to the Product Backlog.



Source: plan.io

Software Engineering Methodologies & Scrum

Pre-Sprint Activities

Product Backlog



Product Backlog Refinement

Split large user story into smaller ones, and clarify requirements and criteria with Definition of Done (DoD).



→ Prioritized PBIs
→ Definition of Done (DoD) and Stakeholder

Sprint Boundary Line

The Sprint (2-4 Weeks)

Sprint timebox

1. Sprint Planning



→ Sprint Backlog

2. Sprint Execution



- Done
- To Do
- Blocked

→ Daily Progress

3. Sprint Review



→ Feedback

4. Sprint Retrospective



→ Process Improvements

Sprint Review PBI update

Release Increment

Definition of Done (DoD)

Post-Sprint Release

9. Relative Sizing vs. Absolute Estimation

De-risking Software Schedules through Empirical Sizing Vectors & Capacity Calibration

Relative Sizing vs. Absolute Estimation

Addressing the inherent failure modes of time-based predictions

Why Absolute Estimation Fails

- Individual Skill Variance: Task completion time varies by 5x-6x between junior and senior engineers.
- Anchoring Bias: Early hourly figures freeze into rigid deadlines, forcing quality trade-offs.
- Cognitive Overhead: Detailed hourly forecasting far in advance consumes excessive energy with minimal accuracy.

The Relative Sizing Paradigm

- Comparative Sizing: Evaluates work against a known baseline item rather than exact hours.
- Vector Decoupling: Separates sheer effort and complexity from time duration.
- Velocity Dependent: Time to complete is calculated purely via empirical team execution capacity.

Dimensions of a Story Point

An abstract, unitless metric encapsulating three critical work dimensions

Complexity

- Algorithmic difficulty and internal logic.
- Data transformations & complex state management.
- Multi-system integration boundaries.

Effort

- Total physical work quantity.
- Writing boilerplate UI vs updating a single config file.
- Repeatability of tasks.

Risk & Uncertainty

- Unknown architectural dependencies.
- Unproven third-party APIs or legacy refactoring.
- Ambiguous database schemas.

The Modified Fibonacci Scale

Non-linear point progression reflecting increasing uncertainty

The Fibonacci Sequence Vector

- Scale Definition: { 1, 2, 3, 5, 8, 13, 20, 40, 100 }.
- Non-Linear Gaps: Gaps widen as numbers grow to reflect escalating uncertainty.
- Subjective Range: Prevents useless debates over minor precision (e.g. 11 vs 12 points).

Sizing Strategy & Refinement

- Small Tasks (1–3 pts): Well-understood, low risk, ready for Sprint execution.
- Medium Tasks (5–13 pts): Moderate scope; manageable within a single Sprint.
- Large Epics (20–100 pts): Too vague or complex; must be broken down during refinement.

10. Consensus-Based Estimation: Planning Poker

Collaborative, consensus-based estimation technique designed to eliminate cognitive bias, eliminate peer influence

Planning Poker

Protocol Objectives

- Prevents Anchoring: Prevents senior team members from swaying junior estimates.
- Encourages Debate: Exposes hidden architectural assumptions early.
- Fosters Ownership: Entire cross-functional team commits to the shared estimate.



Source: doasync.com

Planning Poker Execution Protocol

Planning Poker: Step-by-Step Execution Protocol

1. Presentation



Product Owner (PO) introduces a PBI. Team clarifies functional boundaries and acceptance criteria.

2. Technical Discussion



Developers debate implementation approaches, dependencies, and architectural impacts.

3. Private Valuation



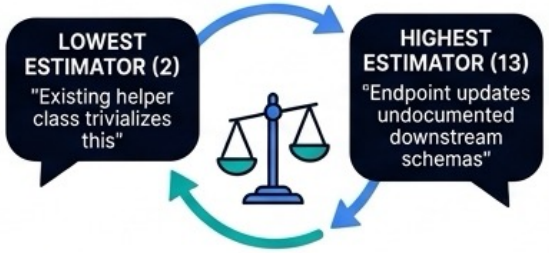
Each developer privately selects a card from their Fibonacci deck. No discussion allowed anchoring bias.

4. Simultaneous Reveal



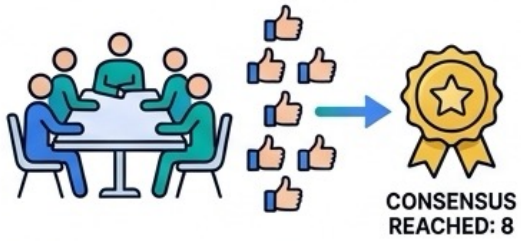
All participants flip their cards at the exact same moment.

5. Divergence Resolution



Team discusses variance, not average. Lowest and highest estimators explain reasoning.

6. Convergence Iteration



Team re-votes until reasonable consensus on a unified estimate is reached.

Source: Generated by Gemini

11. Velocity & Mathematical Forecasting

Velocity is a metric measuring the amount of work a Scrum Team completes during a single Sprint expressed as the sum of Story Points

Sprint Velocity Mechanics

Quantifying delivered business value through empirical output

Velocity Core Concepts

- Formula: Sprint Velocity = Sum of completed Story Points meeting Definition of Done (DoD).
- Binary Rule: All-or-nothing evaluation. A 90% completed 8-point story contributes 0 points to Velocity.
- Uncompleted Items: Remain in Backlog for re-estimation or future Sprints.

Team-Specific Baseline

- Team-Bound Metric: Velocity is tied exclusively to a specific team's baseline and culture.
- No Cross-Team Comparison: A team delivering 50 pts is not inherently better than one delivering 20 pts.
- Stability Goal: Strive for predictable, consistent velocity across Sprints.

Mathematical Forecasting with Velocity

Empirical release planning grounded in historical data

Mean Velocity Calculation

- Formula: Mean Velocity $\tilde{V}$= (Sum of V_i across $\mathcal{N}$ observed Sprints) / $\mathcal{N}$.

$$\bar{V} = \frac{\sum_{i=1}^{N} V_i}{N}$$

- Baseline Threshold: Requires 3 to 5 Sprints of historical data to yield reliable forecasts.
- Empirical Security: Replaces speculative dates with data-driven projections.

Example

- Total Backlog Scope: 150 total Story Points remaining.
- Historical Baseline: Mean Velocity $\tilde{V}$ = 25 points / Sprint.
- Estimated Duration: 150 / 25 = 6 Sprints remaining to complete release.

Sprint Burndown Visual Analytics

Real-time progress tracking against a linear completion trajectory

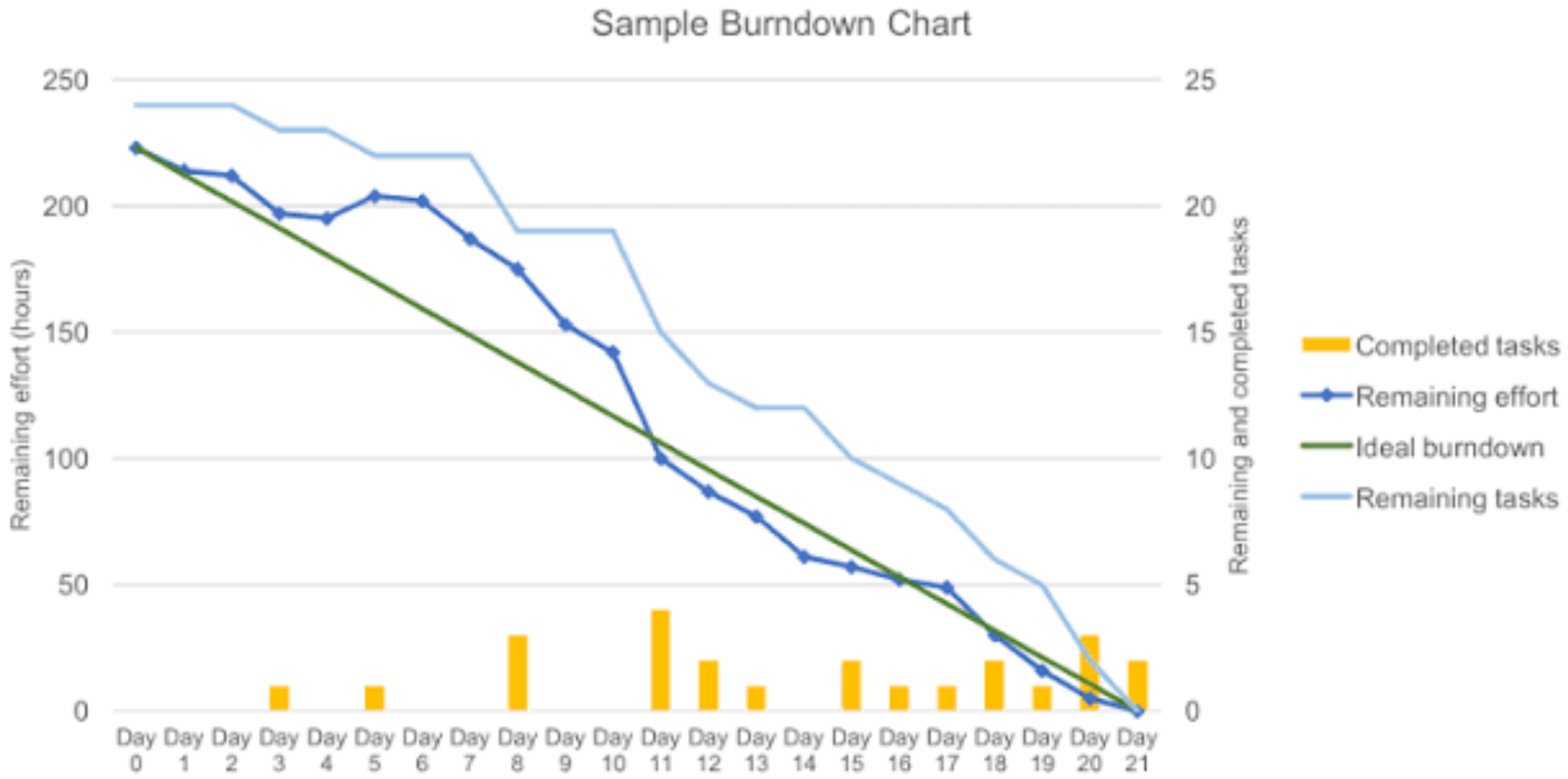
Chart Structure & Axes

- Vertical Axis: Remaining Story Points or tasks in the Sprint Backlog.
- Horizontal Axis: Working days within the Sprint timeframe.
- Ideal Burndown Line: Straight reference baseline from day 0 to zero remaining effort.

Performance Signals

- Above Ideal Line: Team running behind schedule; potential bottleneck or hidden complexity.
- Below Ideal Line: Team completing work faster than planned; opportunity to pull in tasks.
- Daily Scrum Tool: Plotted daily to trigger immediate process adaptation.

Sample Burndown Chart



Source: Wikipedia

Questions?